

Clase 05 - Repaso I1

Profesor: **IIC2343 - Arquitectura de Computadores**
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl

Resumen de las clases pasadas

**¿Qué es un
computador?**

Introducción:

- Un computador lo definimos como una **máquina programable que ejecuta programas.**
- Para programar necesitamos:
 - Datos: números (enteros, reales) , texto, imágenes, etc
 - Operaciones: suma, resta, multiplicación, división, etc
 - Variables: simples, arreglos
 - Control de flujo: comparaciones, manejo de ciclos-

Representación numérica de enteros

Sistema posicional de base genérica

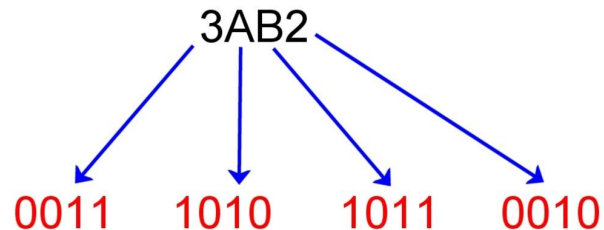
- s : símbolo
- k : = Posición del símbolo en la secuencia, siendo 0 la posición del extremo derecho.
- b : **base**
- n : cantidad de símbolos en la secuencia
- Notación típica: $(\text{---})_b$

$$\sum_{k=0}^{n-1} s_k \times b^k$$

Método de conversión: binaria hacia hexa

- Ocuparemos un método aprovechando concatenación
- Agrupamos los términos numéricos para obtener el resultado

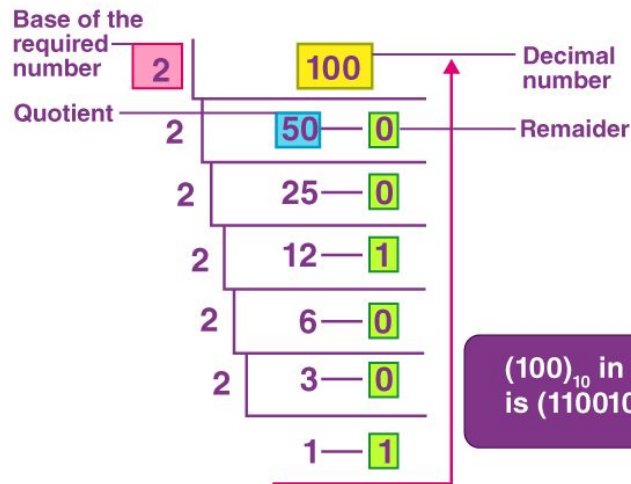
Converting Hex to Binary



$$3AB2_{16} = 11101010110010_2$$

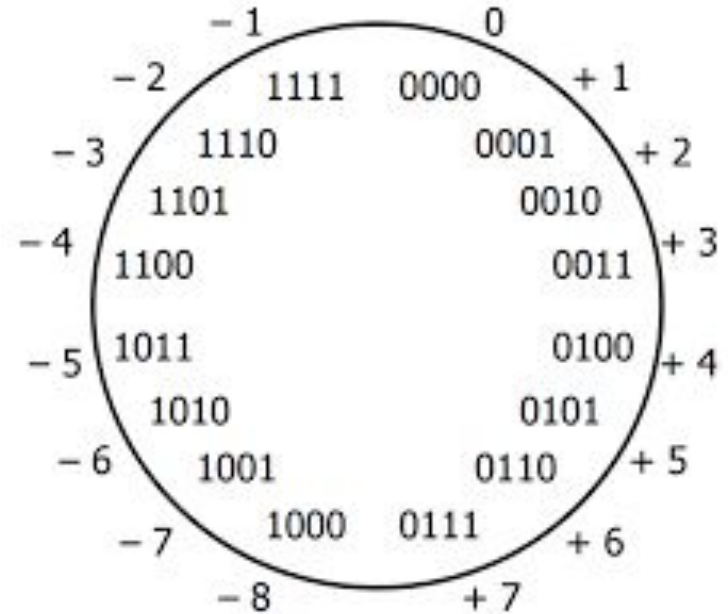
Método de conversión: decimal hacia binario

- Se obtiene el resto entre el número en base decimal y el divisor 2.
- Se obtiene el resto entre el número en base decimal y el divisor 2.
- Para obtener el siguiente símbolo de la secuencia, realizar la misma operación con el resultado de la división entera del número



Complemento 2 (C2)

- Sumar una unidad al complemento al C1
- Ahora el cero es intuitivo
- **Contra:** Tenemos una representación desbalanceada
- **Overflow:** Si una operación aritmética resulta en un valor no representable, nos dará un valor erróneo



string	unsigned	sign & magnitude	1's complement	2's complement
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	4	4	4	4
0101	5	5	5	5
0110	6	6	6	6
0111	7	7	7	7
1000	8	-0	-7	-8
1001	9	-1	-6	-7
1010	10	-2	-5	-6
1011	11	-3	-4	-5
1100	12	-4	-3	-4
1101	13	-5	-2	-3
1110	14	-6	-1	-2
1111	15	-7	-0	-1

Ejercicios

- (a) (2 ptos.) En un videojuego, usted ha logrado acumular un total de 127 vidas. Al seguir jugando, usted adquiere una vida adicional, pero comete un error y pierde una de ellas. En este escenario, puede leer en la pantalla el texto “**GAME OVER**”, pero no entiende cómo perdió considerando el total de vidas que tenía acumulado. A partir de los contenidos del curso en torno a representaciones numéricas, explique por qué podría haber sucedido la situación antes descrita.
- (c) (2 ptos.) A partir de la fórmula de transformación de bases:

$$\sum_{k=0}^{n-1} s_k \times b^k$$

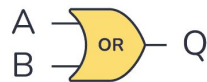
Justifique por qué funciona el algoritmo de conversión directa de base hexadecimal ($b = 16$) a base binaria ($b = 2$) visto en clases.

Circuitos Digitales

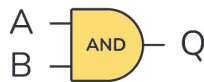
Compuertas lógicas



A	Q
0	1
1	0



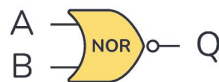
A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0



A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0



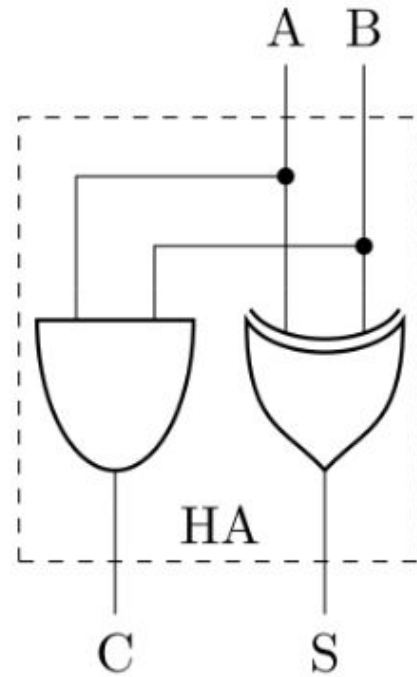
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

Algebra booleana

Equivalence	Name of Identity
$p \wedge T \equiv p$ $p \vee F \equiv p$	Identity Laws ☐
$p \wedge F \equiv F$ $p \vee T \equiv T$	Domination Laws ☐
$p \wedge p \equiv p$ $p \vee p \equiv p$	Idempotent Laws
$\neg(\neg p) \equiv p$	Double Negation Law
$p \wedge q \equiv q \wedge p$ $p \vee q \equiv q \vee p$	Commutative Laws
$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ $(p \vee q) \vee r \equiv p \vee (q \vee r)$	Associative Laws
$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$	Distributive Laws
$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$	De Morgan's Laws ☐
$p \wedge (p \vee q) \equiv p$ $p \vee (p \wedge q) \equiv p$	Absorption Laws
$p \wedge \neg p \equiv F$ $p \vee \neg p \equiv T$	Negation Laws

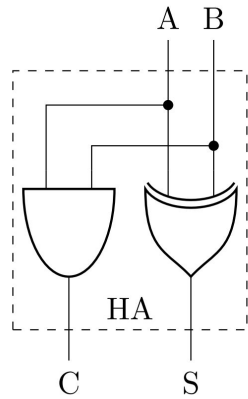
Half Adder

<i>A</i>	<i>B</i>	<i>S</i>	<i>C</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

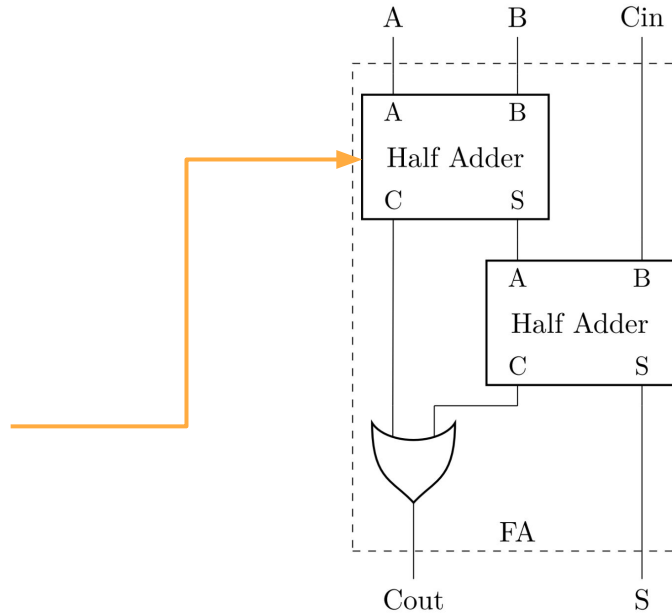


(a) Half Adder

Full Adder

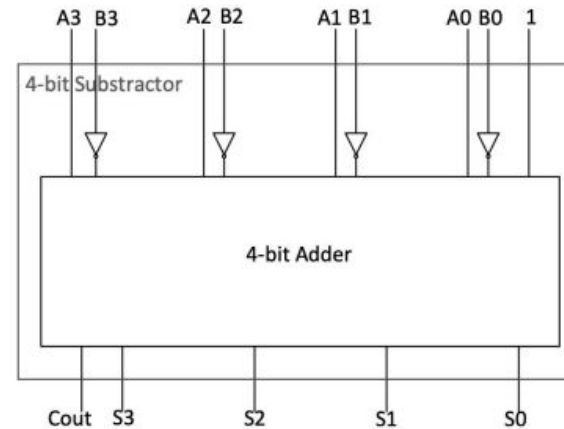
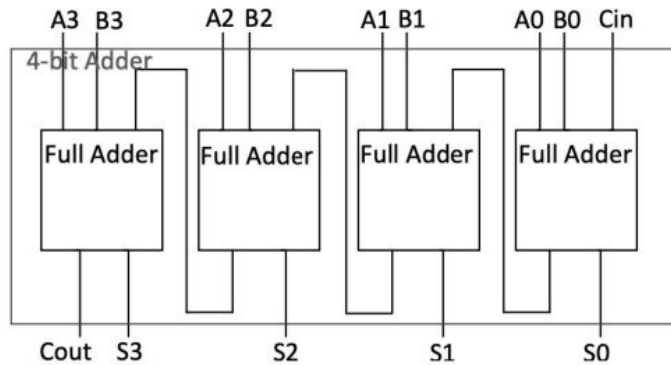


(a) Half Adder



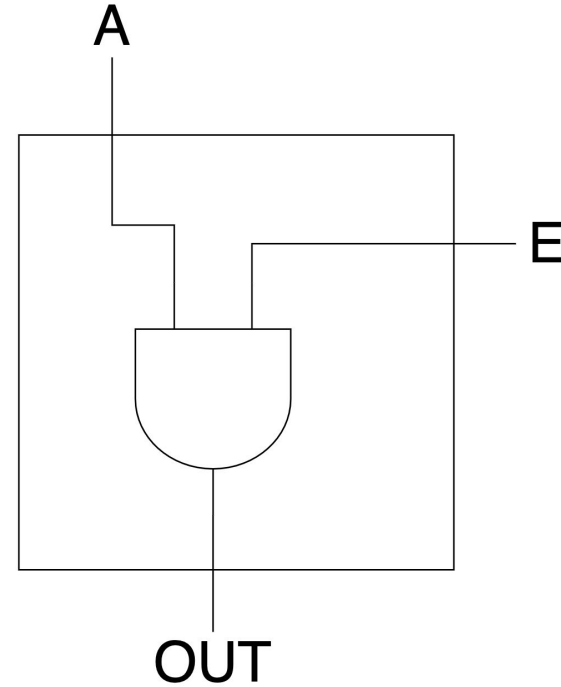
(b) Full Adder

Sumador/Restador



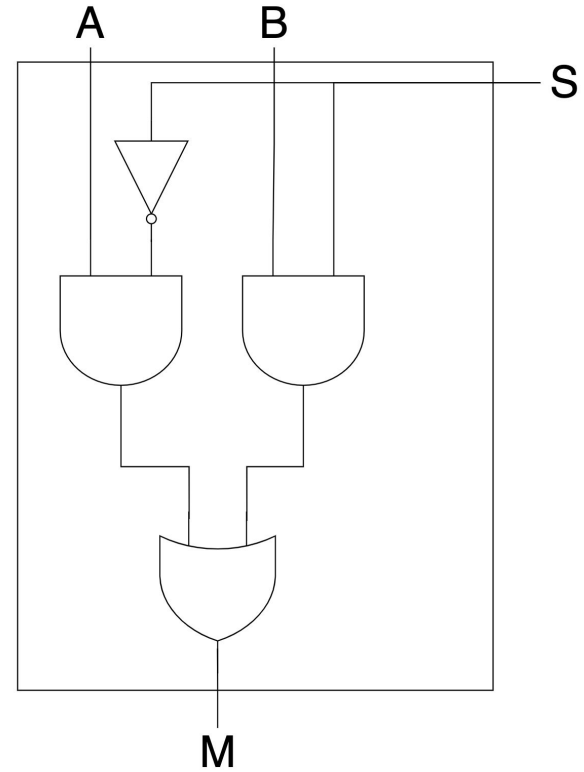
Circuitos de control - Enabler

OUT	E
0	0
A	1



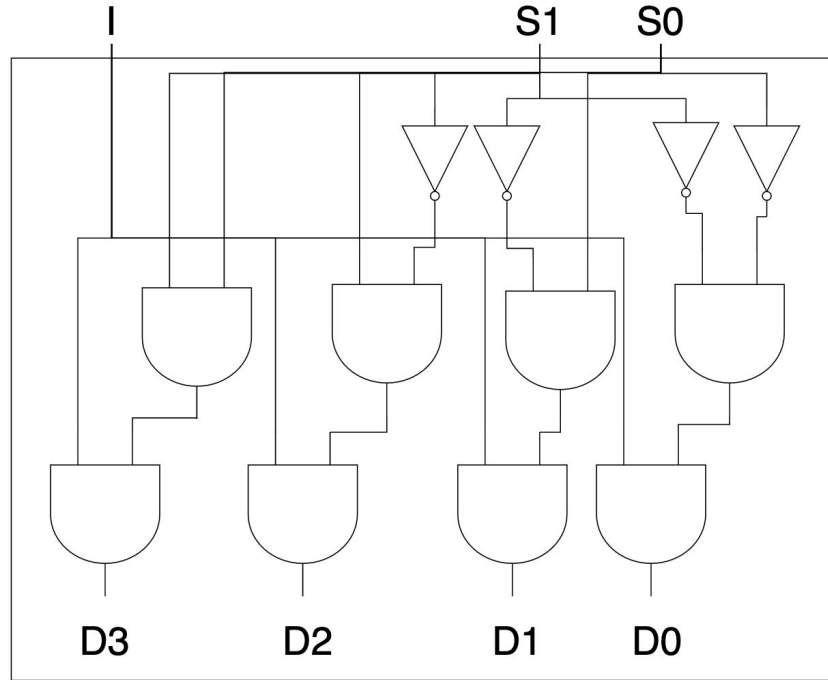
Circuitos de control - Multiplexor

S	M
0	A
1	B



Circuitos de control - Demux

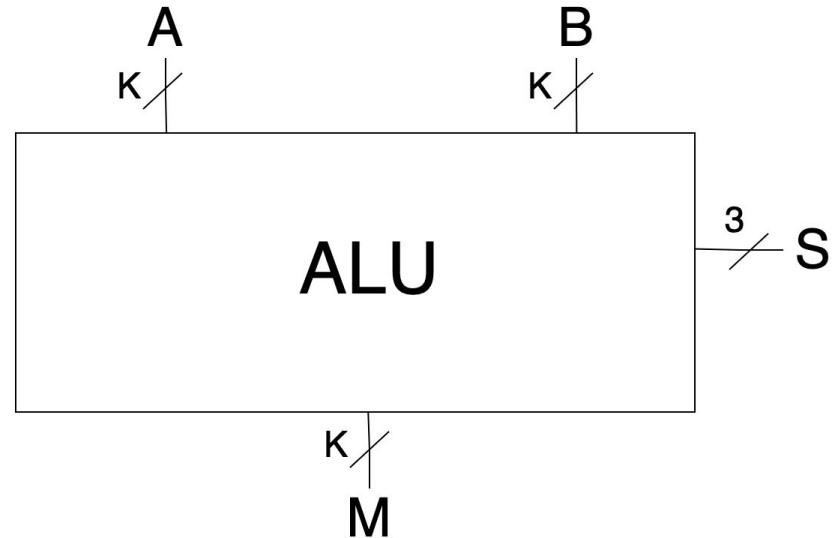
S1	S0	Output
0	0	D0 = I
0	1	D1 = I
1	0	D2 = I
1	1	D3 = I



Unidad Aritmética Lógica: ALU

- Tabla de valores:

S2	S1	S0	M
0	0	0	Suma
0	0	1	Resta
0	1	0	And
0	1	1	Or
1	0	0	Not
1	0	1	Xor
1	1	0	Shift left
1	1	1	Shift right



Ejercicios

- (a) (3 ptos.) Usted se encuentra desarrollando el sistema de combates para un nuevo videojuego. Para optimizar el uso de memoria, ha decidido implementar un sistema de tipos reducido utilizando solo 2 bits. Para mantener un estándar, los tipos se han asignado en estricto orden alfabético: **Fantasma (00)**, **Lucha (01)**, **Psíquico (10)** y **Siniestro (11)**. Durante un turno, el sistema compara el tipo del monstruo que ataca (A_1, A_0) con el tipo del monstruo que defiende (D_0), que solo puede ser fantasma o lucha. La efectividad del ataque se rige por la siguiente tabla:

Ataca \ Defiende	0 (Fantasma)	1 (Lucha)
00 (Fantasma)	$\times 2$	$\times 1$
01 (Lucha)	$\times 1/2$	$\times 1$
10 (Psíquico)	$\times 1$	$\times 2$
11 (Siniestro)	$\times 2$	$\times 1/2$

Dependiendo del resultado de la pelea, la pantalla del juego debe encender distintas señales (R_1 a R_3) para mostrar un mensaje de texto al jugador, definidos de la siguiente forma:

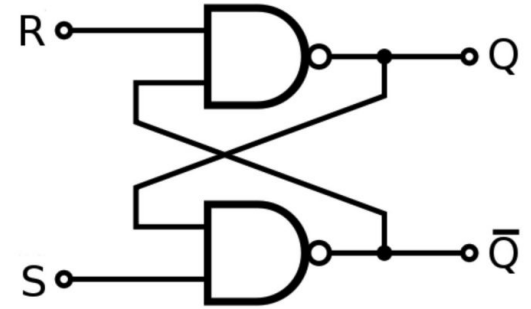
- R_1 : El ataque es **Neutral** ($\times 1$).
- R_2 : El ataque es **Superefectivo** ($\times 2$).
- R_3 : El ataque es **Poco efectivo** ($\times 1/2$).

A partir de este contexto, diseñe para cada salida R_i de la pantalla, un circuito combinacional que reciba las señales de entrada del atacante (A_1, A_0) y del defensor (D_0) y compute si el mensaje R_i debe mostrarse (1) o no (0). Expresé claramente las funciones lógicas utilizadas. Puede, si lo desea, diseñar un único circuito para computar todas las salidas simultáneamente, aprovechando compuertas compartidas.

Almacenamiento de Datos

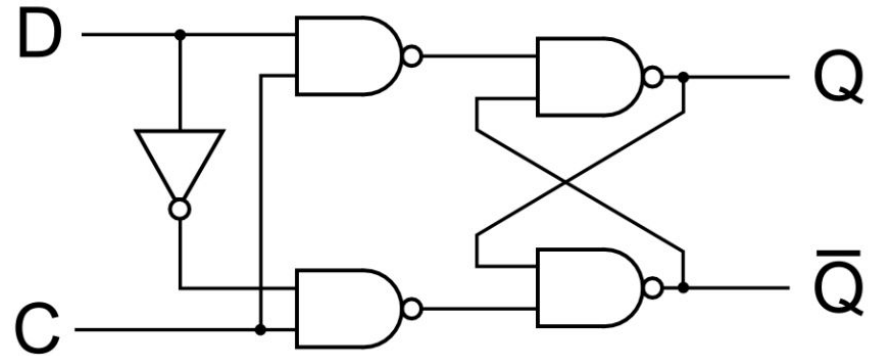
Almacenamiento de datos: Latch RS

S	R	Q(t+1)
0	0	-
0	1	0
1	0	1
1	1	Q(t)



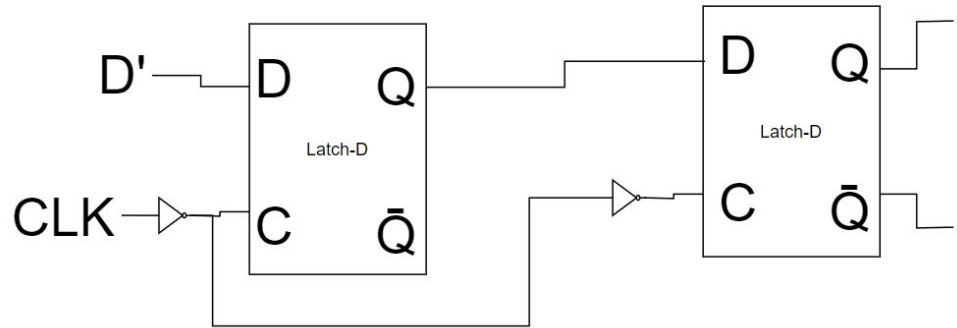
Almacenamiento de datos: Latch D

C	D	Q(t+1)
0	0	Q(t)
0	1	Q(t)
1	0	0
1	1	1



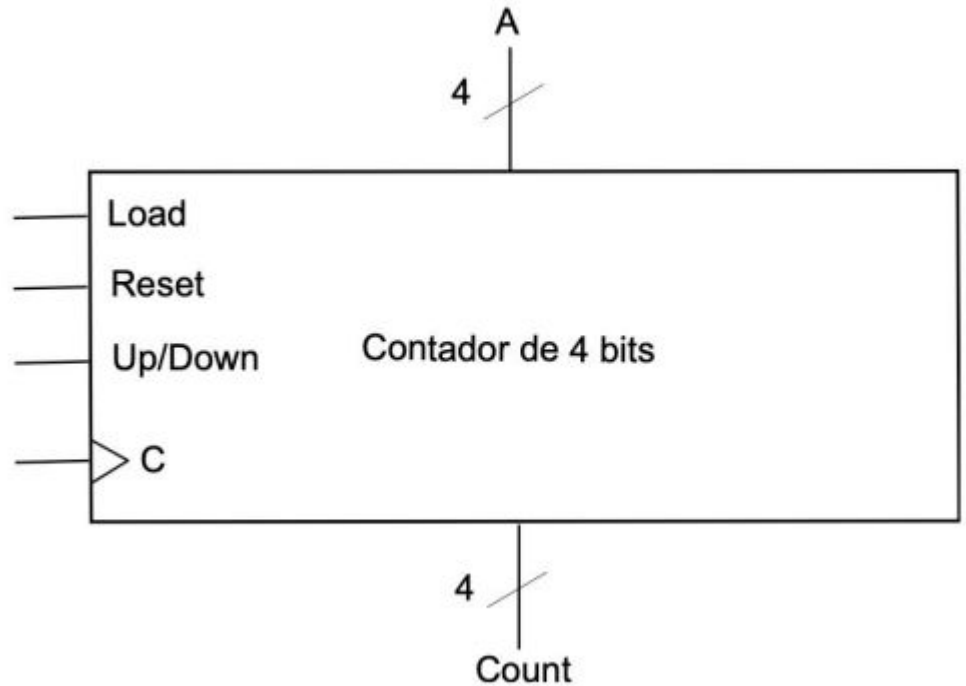
Almacenamiento de datos: Flip-Flop D

C	D	Q(t+1)
↑	0	0
↑	1	1
*	*	Q(t)

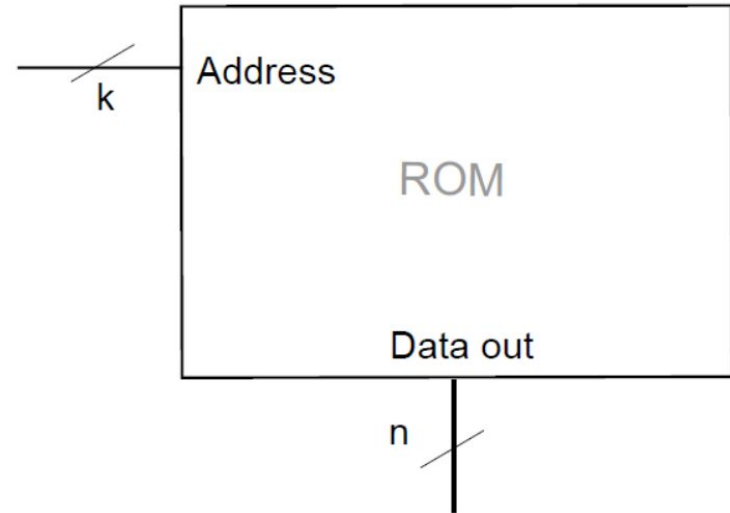
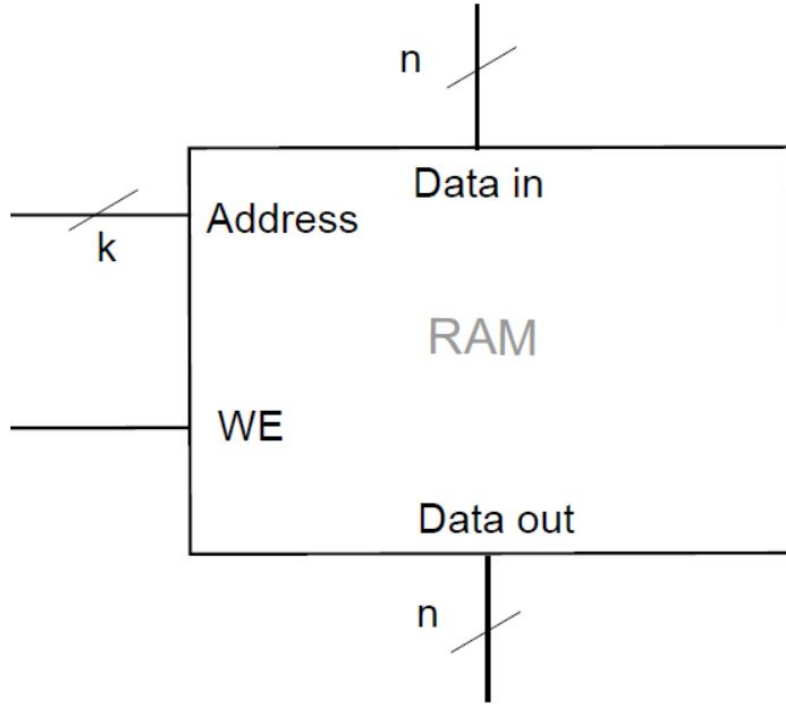


Almacenamiento de datos: Registro Contador

- Con lo anterior se puede generalizar para tener un registro contador
- Una señal Up indicando que el valor almacenado se suma uno
- Una señal Down indicando que el valor almacenado se resta uno



Almacenamiento de datos: Memorias



Ejercicios

- (b) (3 ptos.) Para aplicar los contenidos del curso en su hogar, decide implementar a través de *hardware* una versión reducida del minijuego “*Bowser’s Big Blast*” de Mario Party 2 TM.



Para su implementación debe diseñar, utilizando **solo** compuertas lógicas, *enablers*, multiplexores y flip-flops, un contador secuencial de 2 bits que decrementa en cada flanco de subida a partir de una señal *Boom* según las siguientes condiciones:

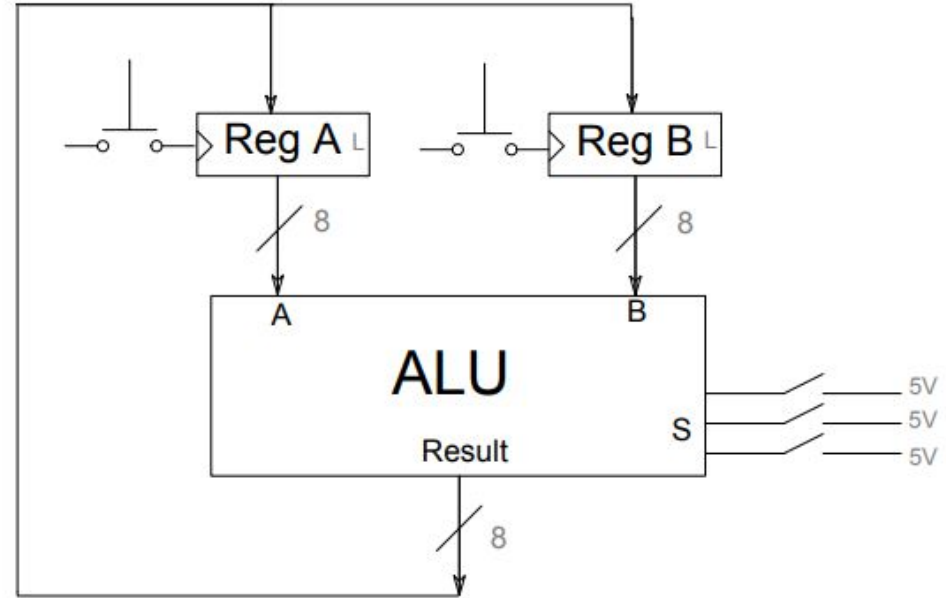
- El contador parte en 3 ($Q_1Q_0 = 11$). Asuma que los flip-flops del contador parten en este estado.
- El contador decrementa en el flanco de subida si, y solo si $Boom = 1$.
- El contador vuelve a 3 ($Q_1Q_0 = 11$) si $Boom = 0$, independiente de su valor actual.

Puede asumir que su contador hace *overflow* en caso de decrementar estando en su mínimo valor, *i.e.* asuma que Q_1Q_0 pasa de 00 a 11 en caso de decrementar con $Boom = 1$.

Programabilidad

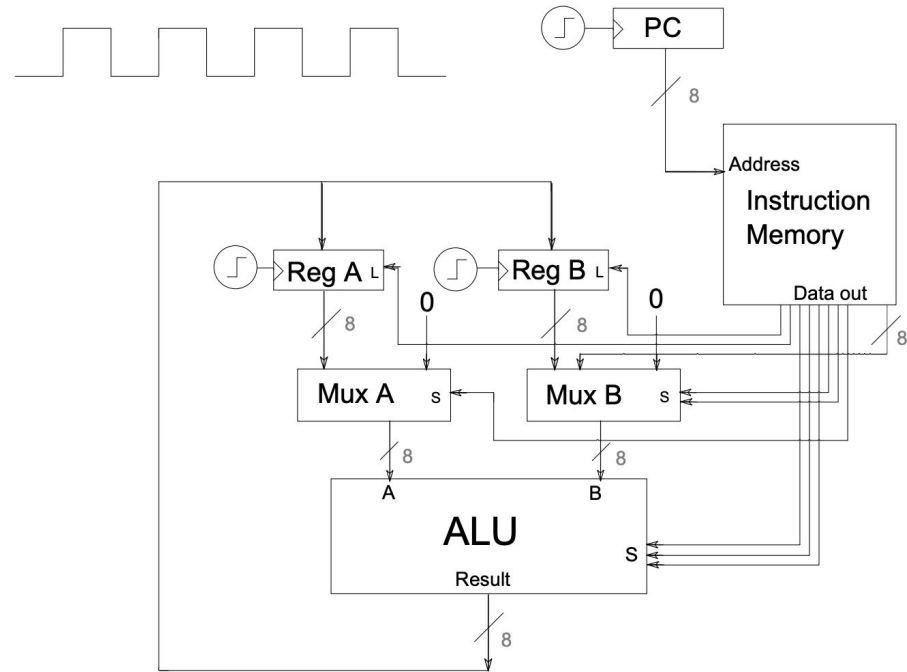
Programabilidad: Calculadora Avanzada

- Ahora tenemos control de lo que cuando almacenar el resultado en los registros
- Notamos que también **tenemos control en la operación** ejecutada en la ALU



Computador básico: Extensión - Literales

- Se debe agregar a la máquina programable es la capacidad de operar con **literales**
- Un **literal** se refiere a un valor numérico que se define explícitamente
- Se extiende la capacidad de la memoria de instrucciones



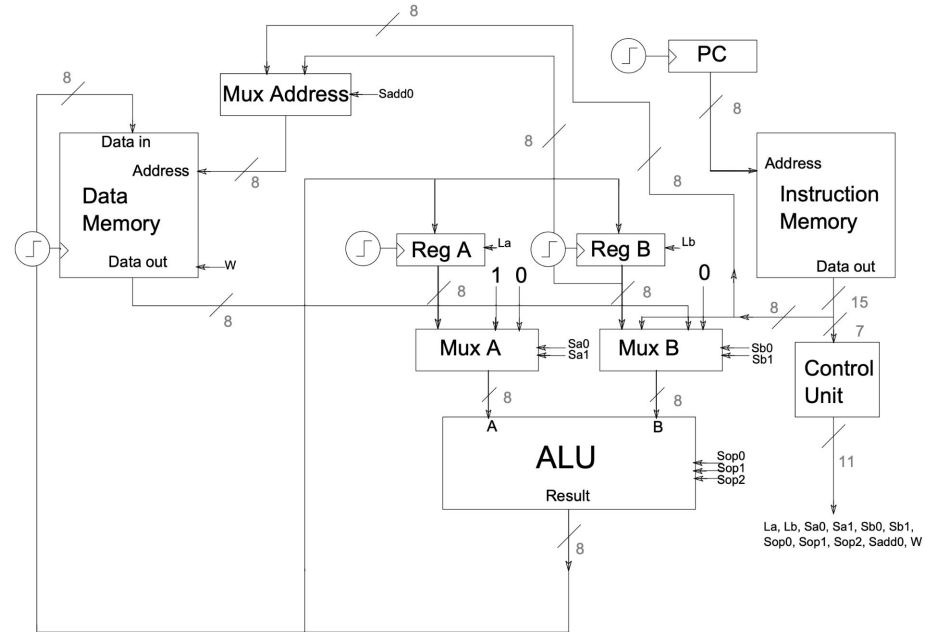
Computador básico: Opcodes

- Usamos *opcodes* (códigos de operación) que definen la combinación de señales de control que ejecuta una instrucción.
- Ahora, falta un **componente** que decodifique *opcodes*.

Opcode	La	Lb	Sa0	Sb0	Sb1	Sop2	Sop1	Sop0	Operación
000000	1	0	1	0	0	0	0	0	A=B
000001	0	1	0	1	1	0	0	0	B=A
000010	1	0	1	0	1	0	0	0	A=Lit
000011	0	1	1	0	1	0	0	0	B=Lit
000100	1	0	0	0	0	0	0	0	A=A+B
000101	0	1	0	0	0	0	0	0	B=A+B
000110	1	0	0	0	1	0	0	0	A=A+Lit
000111	1	0	0	0	0	0	0	1	A=A-B
001000	0	1	0	0	0	0	0	1	B=A-B
001001	1	0	0	0	1	0	0	1	A=A-Lit
001010	1	0	0	0	0	0	1	0	A=A and B
001011	0	1	0	0	0	0	1	0	B=A and B
001100	1	0	0	0	1	0	1	0	A=A and Lit
001101	1	0	0	0	0	0	1	1	A=A or B
001110	0	1	0	0	0	0	1	1	B=A or B
001111	1	0	0	0	1	0	1	1	A=A or Lit
010000	1	0	0	0	0	1	0	0	A=not A
010001	0	1	0	0	0	1	0	0	B=not A
010010	1	0	0	0	0	1	0	1	A=A xor B
010011	0	1	0	0	0	1	0	1	B=A xor B
010100	1	0	0	0	1	1	0	1	A=A xor Lit
010101	1	0	0	0	0	1	1	0	A=shift left A
010110	0	1	0	0	0	1	1	0	B=shift left A
010111	1	0	0	0	0	1	1	1	A=shift right A
011000	0	1	0	0	0	1	1	1	B=shift right A

Computador básico: Direcccionamiento

- **Direcccionamiento Directo:** Se indica la dirección de memoria con un literal
- **Direcccionamiento Indirecto:** Se indica la dirección de memoria con el valor de un registro



Computador básico: Variables en Assembly

- Existirá un segmento **DATA** donde manejaremos las variables
- Siempre luego del segmento de **DATA** se tendrá el segmento de **CODE** donde existirá las instrucciones

Dirección	Label	Instrucción/Dato
DATA:		
0x00	var0	Dato 0
0x01	var1	Dato 1
0x02	var2	Dato 2
0x03		Dato 3
0x04		Dato 4
CODE:		
0x00		Instrucción 0
0x01		Instrucción 1
0x02		Instrucción 2
0x03		Instrucción 3
0x04		Instrucción 4

Ejercicios

Pregunta 3: Computador básico con saltos (6 ptos.)

En los ítems (a), (b) y (c), se le describirán instrucciones que deberá implementar en el computador básico a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida (“-” si no afecta). Debe realizar todas las modificaciones de *hardware* en un solo diagrama. Además, puede implementar instrucciones sin modificaciones si es que la arquitectura se lo permite, indicando las señales de control necesarias. **Cada instrucción debe ejecutarse en un ciclo.**

(a) (1 pto.) **SUB** B,B,A. Almacena en el registro B el resultado de la operación $B - A$.

(b) (1 pto.) **ADD** (B),Lit. Almacena en Mem[B] el resultado de la operación $\text{Mem}[B] + \text{Lit}$.

Salto

Salto Incondicional: Instrucciones (ejemplo)

- La limitante es el hecho de **no iterar**
- Las instrucciones que nos permiten cambiar la dirección de código las llamaremos **saltos**
- En particular en el ejemplo tenemos un **salto incondicional**

Dirección	Instrucción	Operandos
0x00	MOV	A,0
0x01	MOV	B,1
0x02	ADD	A,B
0x03	ADD	B,A
0x04	JMP	0x02

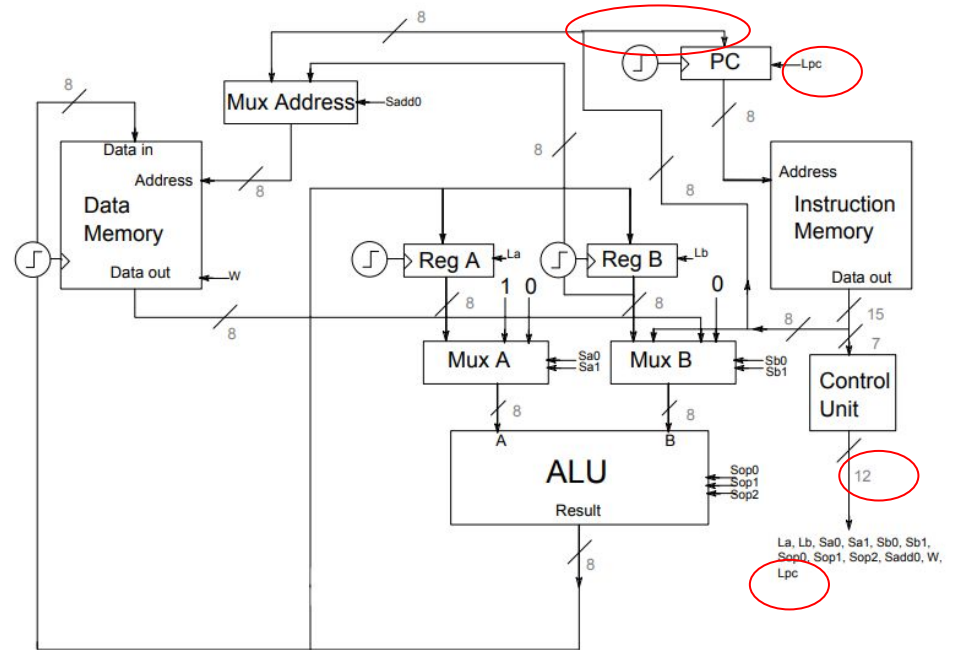
Salto Incondicional: Label

- Es un **indicador** que se puede agregar en una línea del código assembly para referirse a la **dirección de memoria** asociada a esa línea

Dirección	Label	Instrucción	Operandos
0x00		MOV	A,0
0x01		MOV	B,1
0x02	start:	ADD	A,B
0x03		ADD	B,A
0x04		JMP	start

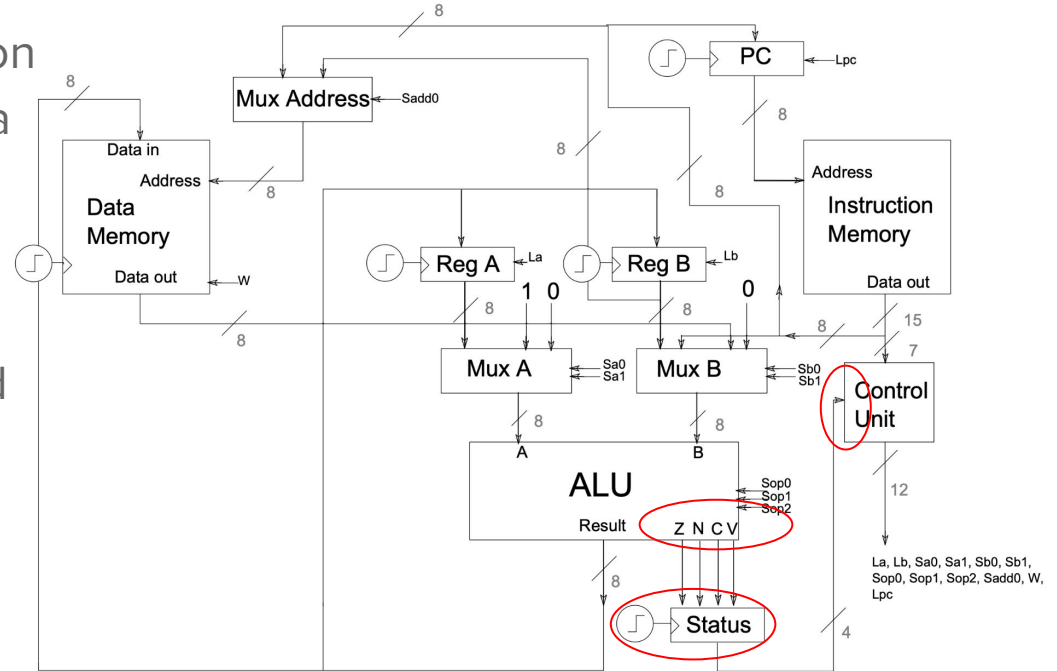
Salto Incondicional: Implementación

- La respuesta es modificar el registro del **Program Counter**
- Esta conexión va desde el literal que viene de la **Instruction Memory**
- Agregamos una nueva señal de control **Lpc** (Load PC)



Salto condicional: Implementación

- Dado que el salto se realiza con las condiciones ocurridas en la operación anterior
- Se necesita un nuevo registro **Status**
- Su salida se conecta la Unidad de Control



Salto Resumen

Instrucción	Operandos	Operación	Condiciones	Ejemplo de uso
CMP	A,B A,Lit	A-B A-Lit		CMP A,0
JMP	Dir	PC = Dir		JMP end
JEQ	Dir	PC = Dir	Z=1	JEQ label
JNE	Dir	PC = Dir	Z=0	JNE label
JGT	Dir	PC = Dir	N=0 y Z=0	JGT label
JLT	Dir	PC = Dir	N=1	JLT label
JGE	Dir	PC = Dir	N=0	JGE label
JLE	Dir	PC = Dir	Z=1 o N=1	JLE label
JCR	Dir	PC = Dir	C=1	JCR label
JOV	Dir	PC = Dir	V=1	JOV label

Ejercicios

En los ítems (a), (b) y (c), se le describirán instrucciones que deberá implementar en el computador básico a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida (“-” si no afecta). Debe realizar todas las modificaciones de *hardware* en un solo diagrama. Además, puede implementar instrucciones sin modificaciones si es que la arquitectura se lo permite, indicando las señales de control necesarias. **Cada instrucción debe ejecutarse en un ciclo.**

(c) (1 pto.) **RESTART**. Carga en el registro PC el valor 0.

Ejercicios

(3 ptos.) Los programas (1) y (2) están escritos en el Assembly del computador básico del curso, con la diferencia de que (2) usa las instrucciones implementadas en los ítems anteriores. Para cada programa, indique los valores de los registros A y B al finalizar la ejecución. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, no se otorgará puntaje.

(1)

```
DATA:
    var1 -1
    var2 -3
CODE:
    MOV A,(var1)
    MOV B,(var2)
    ADD A,B
    JLT case_1
    CMP A,B
    JGT case_2
    JMP case_3
case_1:
    INC B
case_2:
    INC B
case_3:
    INC B
end:
```

Ejercicios

(3 ptos.) Los programas (1) y (2) están escritos en el Assembly del computador básico del curso, con la diferencia de que (2) usa las instrucciones implementadas en los ítems anteriores. Para cada programa, indique los valores de los registros A y B al finalizar la ejecución. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, no se otorgará puntaje.

(2)

```
DATA:
arr 9
    6
index 0
len 2
target 7
diff 127
res -1
CODE:
MOV A,(index)
MOV B,(len)
CMP A,B
JEQ end
MOV B,arr
ADD B,A
MOV A,(B)
MOV B,(target)
SUB B,B,A
JGE continue_1
MOV A,0
SUB A,B
MOV B,A
continue_1:
    MOV A,(diff)
    CMP A,B
    JLE continue_2
    MOV (diff),B
    MOV A,(index)
    MOV (res),A
continue_2:
    MOV B,index
    ADD (B),1
    RESTART
end:
    MOV A,(diff)
    MOV B,(res)
```

Clase 05 - Repaso I1

Profesor: **IIC2343 - Arquitectura de Computadores**
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl